5 栈与队列
5.1基础知识
5.1.1 理论基础
5.1.2 常用函数
Stack（栈）常用函数
Queue（队列）常用函数
双端队列
5.2 经典题目
5.2.1 基本操作
例题：leetcode 232 - 用栈实现队列
例题：leetcode 225 - 用队列实现栈
5.2.2 重复匹配
例题：leetcode 20 - 有效的括号
例题：leetcode 1047 - 删除字符串中的所有相邻重复项
例题：leetcode 150 - 逆波兰表达式求值
5.2.3 双端队列
例题：leetcode 239 - 滑动窗口的最大值
5.2.4 优先队列/小（大）顶堆
例题：leetcode 347 - 前k个高频元素

5 栈与队列

5.1基础知识

5.1.1 理论基础

- 栈和队列的原理应该很熟悉了，队列是先进先出，栈是先进后出。
- 首先大家要知道 栈和队列是STL（C++标准库）里面的两个数据结构。那么来介绍一下，三个最为普遍的STL版本：
  1. HP STL 其他版本的C++ STL，一般是以HP STL为蓝本实现出来的，HP STL是C++ STL的第一个实现版本，而且开放源代码。
  2. P.J.Plauger STL 由P.J.Plauger参照HP STL实现出来的，被Visual C++编译器所采用，不是开源的。
  3. SGI STL 由Silicon Graphics Computer Systems公司参照HP STL实现，被Linux的C++编译器GCC所采用，SGI STL是开源软件，源码可读性甚高。
- 接下来介绍的栈和队列也是SGI STL里面的数据结构， 知道了使用版本，才知道对应的底层实现。
  - 栈提供push 和 pop 等等接口，所有元素必须符合先进后出规则，所以栈不提供走访功能，也不提供迭代器(iterator)。 不像是set 或者map 提供迭代器iterator来遍历所有元素。
- **栈底层容器是可插拔的（也就是说我们可以控制使用哪种容器来实现栈的功能）。**
  - 所以STL中栈往往不被归类为容器，而被归类为container adapter（容器适配器）。
- 那么问题来了，STL 中栈是用什么容器实现的?
- 从下图中可以看出，栈的内部结构，栈的底层实现可以是vector，deque，list 都是可以的， 主要就是数组和链表的底层实现。
- **我们常用的SGI STL，如果没有指定底层实现的话，默认是以deque为缺省情况下栈的底层结构。**
  - deque是一个双向队列，只要封住一端，只开通另一端就可以实现栈的逻辑了。
- 我们也可以指定vector为栈的底层实现，初始化语句如下：

```
std::stack<int, std::vector<int> > third;  // 使用vector为底层容器的栈
```

- 刚刚讲过栈的特性，对应的队列的情况是一样的。
    - 队列中先进先出的数据结构，同样不允许有遍历行为，不提供迭代器, **SGI STL中队列一样是以deque为缺省情况下的底部结构。**
- 也可以指定list 为起底层实现，初始化queue的语句如下：

```
std::queue<int, std::list<int>> third; // 定义以list为底层容器的队列
```

- 所以STL 队列也不被归类为容器，而被归类为container adapter（ 容器适配器）。

5.1.2 常用函数

在C++中，`stack`（栈）和`queue`（队列）是常用的容器适配器，它们基于其他容器（默认为`deque`）实现，提供了特定的数据访问规则。以下是它们的常用函数汇总：

Stack（栈）常用函数

- **`stack<T>`**：构造一个空栈（可指定底层容器，如`vector`或`list`）。
- **`empty()`**：判断栈是否为空，返回`bool`值。
- **`size()`**：返回栈中元素个数。
- **`top()`**：返回栈顶元素的引用（可读可写）。
- **`push(val)`**：将元素压入栈顶。
- **`pop()`**：弹出栈顶元素（无返回值，需确保栈非空）。
- **`emplace(args...)`**：在栈顶构造并插入元素（原地构造，避免拷贝）。
- **`swap(other)`**：与另一个栈交换内容。

Queue（队列）常用函数

- **`queue<T>`**：构造一个空队列（可指定底层容器，如`list`，通常不建议用`vector`）。
- **`empty()`**：判断队列是否为空。
- **`size()`**：返回队列中元素个数。
- **`front()`**：返回队头元素的引用。
- **`back()`**：返回队尾元素的引用。
- **`push(val)`**：将元素加入队尾。
- **`pop()`**：弹出队头元素（无返回值，需确保队列非空）。
- **`emplace(args...)`**：在队尾构造并插入元素。
- **`swap(other)`**：与另一个队列交换内容。

注意事项：

1. **底层容器**：两者默认使用`deque`，但可指定为`vector`或`list`（需满足特定操作要求）。
2. **访问限制**：均不支持迭代器遍历和随机访问（如`[]`运算符）。
3. **函数行为**：`pop()`仅删除元素不返回值，需先通过`top()`/`front()`获取值。
4. **扩展类型**：`priority_queue`（优先队列）是常见变体，默认大顶堆，支持`top()`、`push()`、`pop()`等。

双端队列

分类	操作	说明	时间复杂度
构造	`deque<T> d;`	默认构造空队列	O(1)

	`deque<T> d(n);`	构造含 n 个默认元素的队列	O(n)
	`deque<T> d(n, val);`	构造含 n 个值为 val 的元素	O(n)
	`deque<T> d(begin, end);`	范围构造（数组/其他容器）	O(n)
	`deque<T> d(std::move(d2));`	移动构造（C++11+）	O(1)
元素访问	`d[i]`	随机访问第 i 个元素（**无越界检查**）	O(1)
	`d.at(i)`	随机访问（**有越界检查，越界抛异常**）	O(1)
	`d.front()`	访问队首元素引用	O(1)
	`d.back()`	访问队尾元素引用	O(1)
容量	`d.empty()`	判空	O(1)
	`d.size()`	返回元素个数	O(1)
	`d.max_size()`	返回理论最大容量	O(1)
	`d.resize(n)`	调整大小为 n（新增默认元素或截断）	O(n)
	`d.resize(n, val)`	调整大小并填充值 val	O(n)
	`d.shrink_to_fit()`	释放未用内存（C++11+，仅为建议）	实现定义
修改（双端核心）	`d.push_front(val)`	队首插入	O(1)（均摊）
	`d.push_back(val)`	队尾插入	O(1)（均摊）
	`d.emplace_front(args...)`	队首原地构造（C++11+）	O(1)
	`d.emplace_back(args...)`	队尾原地构造（C++11+）	O(1)
	`d.pop_front()`	删除队首元素	O(1)
	`d.pop_back()`	删除队尾元素	O(1)
	`d.insert(pos, val)`	在 pos 位置插入	O(n)
	`d.insert(pos, n, val)`	插入 n 个值	O(n)
	`d.insert(pos, begin, end)`	插入区间元素	O(n)
	`d.erase(pos)`	删除 pos 位置元素	O(n)
	`d.erase(begin, end)`	删除区间元素	O(n)
	`d.swap(d2)`	交换两个 deque（仅指针操作）	O(1)
	`d.clear()`	清空所有元素	O(n)
迭代器	`d.begin()` / `d.end()`	正向迭代器	O(1)
	`d.cbegin()` / `d.cend()`	常量迭代器（只读，C++11+）	O(1)
	`d.rbegin()` / `d.rend()`	反向迭代器	O(1)

5.2 经典题目

5.2.1 基本操作

例题：leetcode 232 - 用栈实现队列

[232. 用栈实现队列]

- 请你仅使用两个栈实现先入先出队列。队列应当支持一般队列支持的所有操作（`push`、`pop`、`peek`、`empty`）：
- 实现 `MyQueue` 类：
  - `void push(int x)` 将元素 x 推到队列的末尾
  - `int pop()` 从队列的开头移除并返回元素
  - `int peek()` 返回队列开头的元素
  - `boolean empty()` 如果队列为空，返回 `true`；否则，返回 `false`

说明：

- 你 **只能** 使用标准的栈操作 —— 也就是只有 `push to top`, `peek/pop from top`, `size`, 和 `is empty` 操作是合法的。

```
class MyQueue {
public:
    stack<int> in,out;
    MyQueue() {}

    void push(int x) {
        in.push(x);
    }

    int pop() {
        if(out.empty()){
            while(!in.empty()){
                out.push(in.top());
                in.pop();
            }
        }
        int x = out.top();
        out.pop();
        return x;
    }

    int peek() {
        if(out.empty()){
            while(!in.empty()){
                out.push(in.top());
                in.pop();
            }
        }
        return out.top();
    }

    bool empty() {
        return in.empty()&&out.empty();
    }
};
```

- 一个合理的思路是用一个输入栈接受输入，一个输出栈来实现先入先出，具体为：输入就直接压入输入栈，输出时，如果out是空的，则把in按顺序压入并清空in，此时out的顺序就是in反过来的，也就是先入先出了。

例题：leetcode 225 - 用队列实现栈

[225. 用队列实现栈]

- 请你仅使用两个队列实现一个后入先出（LIFO）的栈，并支持普通栈的全部四种操作（`push`、`top`、`pop` 和 `empty`）。
- 实现 `MyStack` 类：
  - `void push(int x)` 将元素 x 压入栈顶。
  - `int pop()` 移除并返回栈顶元素。
  - `int top()` 返回栈顶元素。
  - `boolean empty()` 如果栈是空的，返回 `true` ；否则，返回 `false` 。

```cpp
class MyStack {
public:
    queue<int> main,copy;

    MyStack() {}

    void push(int x) {
        main.push(x);
    }

    int pop() {
        int n=main.size();
        for(int i=0;i<n-1;i++){
            copy.push(main.front());
            main.pop();
        }
        int res = main.front();
        main.pop();
        while(!copy.empty()){
            main.push(copy.front());
            copy.pop();
        }
        return res;
    }

    int top() {
        int n=main.size();
        for(int i=0;i<n-1;i++){
            copy.push(main.front());
            main.pop();
        }
        int res = main.front();
        copy.push(main.front());
        main.pop();
        while(!copy.empty()){
            main.push(copy.front());
            copy.pop();
        }
        return res;
    }

    bool empty() {
        return main.empty()&&copy.empty();
    }
};
```

```

```

- 和上题的思路略有不同：这道题需要用一个队列来做操作，另一个负责备份就行了。
    - 一个很小的细节：注意到在for中如果用main.size()是不对的，因为它会随时变化，所以得先用一个整数存储下来。

5.2.2 重复匹配

例题：leetcode 20 - 有效的括号

[20. 有效的括号]

- 给定一个只包括 `'('`，`')'`，`'{'`，`'}'`，`'['`，`']'` 的字符串 `s`，判断字符串是否有效。
- 有效字符串需满足：
    1. 左括号必须用相同类型的右括号闭合。
    2. 左括号必须以正确的顺序闭合。
    3. 每个右括号都有一个对应的相同类型的左括号。

```
class Solution {
public:
    bool isValid(string s) {
        stack<char> kuohao;
        int n=s.size();
        for(int i=0;i<n;i++){
            if(s[i]=='('||s[i]=='['||s[i]=='{') kuohao.push(s[i]);
            else{
                if(kuohao.empty()) return false;
                if(kuohao.top()=='('&&s[i]!=')') return false;
                if(kuohao.top()=='['&&s[i]!=']') return false;
                if(kuohao.top()=='{'&&s[i]!='}') return false;
                kuohao.pop();
            }
        }
        if(kuohao.empty()) return true;
        else return false;
    }
};
```

- 经典的题目，核心思想是消去：每次入栈左括号，要完美匹配只需要输入右括号时，和最近的一个左括号能消去。

例题：leetcode 1047 - 删除字符串中的所有相邻重复项

[1047. 删除字符串中的所有相邻重复项]

- 给出由小写字母组成的字符串 `s`，**重复项删除操作**会选择两个相邻且相同的字母，并删除它们。
- 在 `s` 上反复执行重复项删除操作，直到无法继续删除。
- 在完成所有重复项删除操作后返回最终的字符串。答案保证唯一。

```
class Solution {
public:
    string removeDuplicates(string s) {
        stack<char> zimu;
        for(int i=0;i<s.size();i++){
            if(zimu.empty()) zimu.push(s[i]);
            else{
                if(zimu.top()==s[i]) zimu.pop();
```

```
                else zimu.push(s[i]);
            }
        }
        string res;
        while(!zimu.empty()){
            res.push_back(zimu.top());
            zimu.pop();
        }
        reverse(res.begin(),res.end());
        return res;
    }
};
```

- 有关于重复匹配、消去的，用栈或者类似的思想都可以很好的解决。

例题：leetcode 150 - 逆波兰表达式求值

[150. 逆波兰表达式求值]

- 给你一个字符串数组 `tokens` ，表示一个根据 [逆波兰表示法] 表示的算术表达式。
- 请你计算该表达式。返回一个表示表达式值的整数。
- **注意：**
    - 有效的算符为 `'+'`、`'-'`、`'*'` 和 `'/'` 。
    - 每个操作数（运算对象）都可以是一个整数或者另一个表达式。
    - 两个整数之间的除法总是 **向零截断** 。
    - 表达式中不含除零运算。
    - 输入是一个根据逆波兰表示法表示的算术表达式。
    - 答案及所有中间计算结果可以用 **32 位** 整数表示。

```
class Solution {
public:
    int evalRPN(vector<string>& tokens) {
        stack<int> poland;
        int n=tokens.size();
        for(int i=0;i<n;i++){
            if(tokens[i]!="+"&&tokens[i]!="-"&&tokens[i]!="*"&&tokens[i]!="/"){
                int num = atoi(tokens[i].c_str());
                poland.push(num);
            }
            if(tokens[i]=="+"){
                int x2=poland.top();poland.pop();
                int x1=poland.top();poland.pop();
                poland.push(x1+x2);
            }
            if(tokens[i]=="-"){
                int x2=poland.top();poland.pop();
                int x1=poland.top();poland.pop();
                poland.push(x1-x2);
            }
            if(tokens[i]=="*"){
                int x2=poland.top();poland.pop();
                int x1=poland.top();poland.pop();
                poland.push(x1*x2);
            }
```

```
            if(tokens[i]=="/"){
                int x2=poland.top();poland.pop();
                int x1=poland.top();poland.pop();
                poland.push(x1/x2);
            }
        }
        return poland.top();
    }
};
```

- 这道题目难点在于转整数，不过好在有现成的公式可以用，一个比较坑的地方的是，注意x1和x2的顺序。

5.2.3 双端队列

例题：leetcode 239 - 滑动窗口的最大值

[239. 滑动窗口最大值]

- 给你一个整数数组 `nums`，有一个大小为 `k` 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 `k` 个数字。滑动窗口每次只向右移动一位。
- 返回 *滑动窗口中的最大值* 。

```
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        deque<int> winmax;
        vector<int> res;

        for(int i=0;i<k;i++){
            while(!winmax.empty()&&nums[i]>=nums[winmax.back()]) winmax.pop_back();
            winmax.push_back(i);
        }
        res.push_back(nums[winmax.front()]);

        for(int i=0;i<nums.size()-k;i++){
            int out=i,in=k+i;
            if(winmax.front()==out) winmax.pop_front();
            while(!winmax.empty()&&nums[in]>=nums[winmax.back()]) winmax.pop_back();
            winmax.push_back(in);
            res.push_back(nums[winmax.front()]);
        }

        return res;
    }
};
```

- 这道题暴力算法为O(n(n-k))，但我们采用双端队列的话可以实现线性复杂度：
- 思想如下：
    - 由于我们需要求出的是滑动窗口的最大值，如果当前的滑动窗口中有两个下标 i 和 j，其中 i 在 j 的左侧（i<j），并且 i 对应的元素不大于 j 对应的元素（nums[i]≤nums[j]），那么会发生什么呢?
    - 当滑动窗口向右移动时，只要 i 还在窗口中，那么 j 一定也还在窗口中，这是 i 在 j 的左侧所保证的。因此，由于 nums[j] 的存在，nums[i] 一定不会是滑动窗口中的最大值了，我们可以将 nums[i] 永久地移除。
    - 因此我们可以使用一个队列存储所有还没有被移除的下标。在队列中，这些下标按照从小到大的顺序被存储，并且它们在数组 nums 中对应的值是严格单调递减的。因为如果队列中有两个相邻的下标，它们对应的值相等或者递增，那么令前者为 i，后者为 j，就对应了上面所说的情况，即 nums[i] 会被移除，这就产生了矛盾。

    - 当滑动窗口向右移动时，我们需要把一个新的元素放入队列中。为了保持队列的性质，我们会不断地将新的元素与队尾的元素相比较，如果前者大于等于后者，那么队尾的元素就可以被永久地移除，我们将其弹出队列。我们需要不断地进行此项操作，直到队列为空或者新的元素小于队尾的元素。
    - 由于队列中下标对应的元素是严格单调递减的，因此此时队首下标对应的元素就是滑动窗口中的最大值。但与方法一中相同的是，此时的最大值可能在滑动窗口左边界的左侧，并且随着窗口向右移动，它永远不可能出现在滑动窗口中了。因此我们还需要不断从队首弹出元素，直到队首元素在窗口中为止。
    - 为了可以同时弹出队首和队尾的元素，我们需要使用双端队列。满足这种单调性的双端队列一般称作「单调队列」。
- 具体实现如下：
    - 每次从后输入，但首先要删除队列尾部的比自己小还比自己序号小的项目；
    - 滑动窗口：每次判断最大项有没有被清除走，清除走就删除了，然后按上述逻辑输入一个数；
    - 输出：上述逻辑保证了队列的头元素永远是该窗口的最大值。

5.2.4 优先队列/小（大）顶堆

例题：leetcode 347 - 前k个高频元素

[347. 前 K 个高频元素]

- 给你一个整数数组 `nums` 和一个整数 `k` ，请你返回其中出现频率前 `k` 高的元素。你可以按 **任意顺序** 返回答案。

```
class Solution {
public:

    class mycomparison {
    public:
        bool operator()(const pair<int, int>& lhs, const pair<int, int>& rhs) {
            return lhs.second > rhs.second;
        }
    };

    vector<int> topKFrequent(vector<int>& nums, int k) {
        vector<int> res(k);
        unordered_map<int,int> mp;
        for(auto &a:nums) mp[a]++;
        priority_queue<pair<int, int>, vector<pair<int, int>>, mycomparison> pq;

        for (unordered_map<int, int>::iterator it = mp.begin(); it != mp.end(); it++) {
            pq.push(*it);
            if (pq.size() > k) {
                pq.pop();
            }
        }

        for (int i = k - 1; i >= 0; i--) {
            res[i] = pq.top().first;
            pq.pop();
        }
        return res;
    }
};
```

- 第一步思路很简单：统计频率，显然用哈希表非常方便，那如何排序前k项呢?
- 一个简单的思路是用小顶堆，其是一个完全二叉树，性质为父节点永远小于等于其孩子节点，每次弹出的top()都是根节点

也就是所有节点中最小的那一项，有了这个性质后我们就很简单能想到，只要能遍历哈希表中的键值对，输入小顶堆，如果小顶堆的大小超过k了，就弹出一个（最小的一项），最后保留的小顶堆就是最大的k项。

- 小顶堆的定义比较复杂：

```
template<
    class T,                          // 元素类型
    class Container = vector<T>,      // 底层容器（默认 vector）
    class Compare = less<T>           // 比较器（默认大顶堆）
> class priority_queue;
```

- 比较器逻辑：

```
class mycomparison {
public:
    bool operator()(const pair<int, int>& lhs, const pair<int, int>& rhs) {
        return lhs.second > rhs.second;  // 核心逻辑
    }
};
```

- 作用：定义元素的“优先级”顺序。
- 返回值含义：
    - operator()(a, b) 返回 true 表示 a 的优先级低于 b，即 b 应该排在 a 前面。
    - 当 lhs 的频率大于 rhs 时，lhs 优先级更低 → 频率小的元素排在堆顶 → 小顶堆。